PART 1 — Create a GitHub Account and Repository (15 minutes)

bash

```
# Install GitHub CLI
sudo apt install gh -y

# Authenticate
gh auth login
# Follow prompts — choose GitHub.com, HTTPS, authenticate via browser

# Create a repository
cd ~/labs/week10/payliteng_secure
gh repo create payliteng-secure \
  --private \
  --source=. \
  --remote=origin \
  --push

# Verify
gh repo view
```

PART 2 — What is GitHub Actions? (15 minutes)

> "GitHub Actions is an automation platform. You write a YAML file that tells GitHub what to do when specific events happen — like pushing code. The YAML file lives inside your repository in the .github/workflows/ folder."

The Structure

yaml

```
# .github/workflows/security.yml

name: Name of the workflow

on:                      # WHEN does this run?
  push:                   # On every push
    branches: [main, develop]
  pull_request:           # On every pull request

jobs:                    # WHAT does it do?
  job-name:               # Name of the job
    runs-on: ubuntu-latest   # Which OS to run on

    steps:                # Steps inside the job
```

```
    - name: Checkout code  # Step 1 — get the code
      uses: actions/checkout@v3

    - name: Run a check    # Step 2 — run a tool
      run: echo "Running security check"
```

PART 3 — Build the First Pipeline (30 minutes)

bash

```
mkdir -p ~/labs/week10/payliteng_secure/.github/workflows

cat > ~/labs/week10/payliteng_secure/.github/workflows/security.yml << 'EOF'
name: PayliteNG Security Pipeline

on:
  push:
    branches: [main, develop, feature/**]
  pull_request:
    branches: [main, develop]

jobs:

  # ── JOB 1 — SECRET DETECTION ──────────────────────────────────────
  secret-detection:
    name: Secret Detection
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3
        with:
          fetch-depth: 0   # Full history for TruffleHog

      - name: TruffleHog Secret Scan
        uses: trufflesecurity/trufflehog@main
        with:
          path: ./
          base: ${{ github.event.repository.default_branch }}
          head: HEAD
          extra_args: --debug --only-verified
```

```
# ── JOB 2 — SAST ──────────────────────────────────────────────────────────

sast-scan:
  name: Static Application Security Testing
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.11'

    - name: Install Bandit
      run: pip install bandit

    - name: Run Bandit SAST Scan
      run: |
        bandit -r . \
          -f json \
          -o bandit_report.json \
          --severity-level medium \
          --confidence-level medium \
          -x ./.git,./tests
      continue-on-error: true

    - name: Display Bandit Results
      run: |
        if [ -f bandit_report.json ]; then
          python3 -c "
        import json
        with open('bandit_report.json') as f:
          data = json.load(f)
        results = data.get('results', [])
        print(f'Issues found: {len(results)}')
        for r in results:
          print(f\"[{r['issue_severity']}] {r['issue_text']}\")
          print(f\"  File: {r['filename']}:{r['line_number']}\")
        "
        fi

    - name: Fail if High Severity Issues
      run: |
```

```
        HIGH=$(python3 -c "
        import json
        with open('bandit_report.json') as f:
          data = json.load(f)
        high = [r for r in data.get('results', []) if r['issue_severity'] == 'HIGH']
        print(len(high))
        ")
        if [ "$HIGH" -gt "0" ]; then
          echo "PIPELINE BLOCKED: $HIGH high-severity issues found"
          exit 1
        fi

# ── JOB 3 — DEPENDENCY SCAN ──────────────────────────────────────────
dependency-scan:
  name: Dependency Vulnerability Scan
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.11'

    - name: Install Safety
      run: pip install safety

    - name: Run Safety Dependency Scan
      run: |
        safety check \
          -r requirements.txt \
          --json \
          -o safety_report.json
      continue-on-error: true

    - name: Display Dependency Results
      run: |
        if [ -f safety_report.json ]; then
          python3 -c "
        import json
        with open('safety_report.json') as f:
          data = json.load(f)
        vulns = data.get('vulnerabilities', [])
```

```
          print(f'Vulnerable packages found: {len(vulns)}')
          for v in vulns:
            print(f\"Package: {v['package_name']} {v['analyzed_version']}\")
            print(f\"CVE: {v.get('cve', 'N/A')} — {v['advisory'][:80]}\")
          "
          fi

      - name: Fail if Critical CVE Found
        run: |
          CRITICAL=$(python3 -c "
          import json
          with open('safety_report.json') as f:
            data = json.load(f)
          high = [v for v in data.get('vulnerabilities', []) if v.get('cvss_score', 0) >= 7.0]
          print(len(high))
          ")
          if [ "$CRITICAL" -gt "0" ]; then
            echo "PIPELINE BLOCKED: Vulnerable dependencies found"
            exit 1
          fi

EOF

git add .github/
git commit -m "Add security pipeline — secret detection, SAST, dependency scan"
git push origin feature/user-authentication
```